

Orientation

What `hathor-core` is, what a full node does, and how one lives — from the moment you type a command to a node that is synced, verifying, and mining.

SUBJECT hathor-core · the whole node

CHAPTER 00 · Orientation

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Full node · Peer-to-peer · DAG · Twisted reactor · RocksDB · Genesis · Consensus · Mining · Sync

Table of Contents

Chapter 0 — Orientation	3
0.1 What Hathor is	3
What hathor-core is, as a program	4
0.2 What a full node does	4
0.3 The life of a node	5
Act I — Startup	5
Act II — Steady state	7
0.4 The module map	8
Domain model — the data and its rules	9
Storage and lookup — where the ledger lives	10
Networking and clients — talking to the world	10
Mining and monetary — minting and securing coins	10
Higher services — built on the base ledger	11
Infrastructure — making it all run	11
The command-line surface	11
0.5 How to read this book	11
Three reading aids	12
The order, and how to navigate	12
Recap	13

Chapter 0 — Orientation

What you'll learn in this chapter

- What Hathi is, and what the `hathi-core` software actually is as a program.
- What a full node is responsible for, in plain terms.
- The **life of a node**: the ordered story of what happens between typing `hathi-cli run_node` and a node that is fully synced, verifying transactions, and mining.
- The **module map**: how the codebase divides itself into packages, what each one is for, and where it sits.
- **How to read this book** — the three content tracks, and the footnote / recap / deep-dive convention used throughout.

This chapter is the map you keep folded in your pocket for the rest of the book. It does not explain how anything works in detail — every later chapter does that. Its job is to give you a mental model of the whole system so that when a later chapter says "the manager hands the vertex to the verification pipeline," you already know what the manager is, what a vertex is, and roughly where verification sits. Read it once now; come back to §0.3 and §0.4 whenever you feel lost.

There is almost no code here on purpose. Code starts in Chapter 1.

0.1 What Hathi is

Hathi is a **cryptocurrency network**: a system that lets people hold and transfer a digital token without any bank, company, or central server being in charge. Like Bitcoin, it is a distributed ledger¹ — a shared record of who owns what — that is maintained by many independent computers around the world, none of which has to trust any other.

What makes Hathi distinct from Bitcoin is the shape of that ledger. Bitcoin organizes its history as a single chain of blocks, one after another. Hathi uses a **DAG**² — a directed acyclic graph — in which individual transactions are themselves part of the data structure, not just bundled inside blocks. Blocks still exist (they are how new coins are minted and how the network agrees on an ordering), but ordinary transactions link directly to other transactions, forming a graph rather than a straight

line. We give the full treatment of why this design exists, and what it buys, in **Chapter 8**. For now, hold this one sentence: **in Hathor, both blocks and transactions are nodes in one graph, and the codebase calls any such node a "vertex."**³

On top of that base ledger, Hathor supports a few things worth naming now so the vocabulary isn't a surprise later:

- **The native token, HTR** — the built-in currency, created by mining.
- **Custom tokens** — users can create their own tokens that ride on the same ledger.
- **Nano-contracts** — Hathor's form of smart contracts⁴: small programs, stored on the ledger, that move tokens according to rules. These are the single largest subsystem in the codebase (Chapter 39).

What `hathor-core` is, as a program

`hathor-core` is the **reference implementation** of a Hathor full node, written in Python. "Reference implementation" means it is the canonical, official program that defines how a correct node behaves; if another team wrote a second Hathor node, they would check their behavior against this one.

Concretely, `hathor-core` is a **long-running server process** — a daemon⁵. You start it from a terminal, and it keeps running indefinitely: talking to other nodes over the network, validating new transactions and blocks as they arrive, storing them on disk, and answering questions from wallets and miners. It is not a website, not a phone app, and not a one-shot script that runs and exits. It is the engine that sits underneath all of those.

When you install it, you get one command-line program, conventionally invoked as `hathor-cli`, which can do many jobs (run a node, generate a wallet, run a miner, dump the database, and more). Running an actual node is just one of those jobs — the subcommand `run_node`. The rest of this book is overwhelmingly about what `run_node` sets in motion.

0.2 What a full node does

The word "full" is doing real work in "full node." A full node is one that keeps and independently verifies the **entire** ledger, trusting no one to have done the checking for it. (There are lighter kinds of participant — wallets that ask a full node for answers rather than storing everything themselves — but `hathor-core` is the full kind.)

A full node has five standing responsibilities. Almost every package in the codebase exists to serve one of them, so it is worth fixing them in mind:

1. **Store the ledger.** Keep every block and transaction, plus derived bookkeeping, on disk, and be able to look any of it up quickly. (Chapters 27–28: storage and indexes.)
2. **Validate everything.** When a new transaction or block arrives, check that it obeys every rule — signatures are valid, no coins are spent twice, proof-of-work is sufficient — before accepting it. A full node never takes another node's word for validity. (Chapter 31: verification.)
3. **Reach consensus.** Decide, by the network's agreed rules, which version of history is the real one when there is any ambiguity (for example, two conflicting transactions). (Chapter 32: consensus.)
4. **Talk to peers.** Find other nodes, connect to them, and exchange data so that everyone converges on the same ledger. This is the **peer-to-peer**⁶ layer, and "catching up" to the rest of the network is called **syncing**⁷. (Chapters 34–35: P2P and sync.)
5. **Serve clients.** Expose interfaces — HTTP APIs, WebSockets, a mining protocol — so that wallets can read balances, miners can fetch work, and operators can monitor the node. (Chapters 30, 36, 37: events/WebSocket, mining.)

Everything else — the command-line surface, configuration, the reactor, the builder, metrics — is infrastructure that exists so these five jobs can be done reliably. Keep this list nearby; in §0.4 you will see that the module groups map almost one-to-one onto these responsibilities.

0.3 The life of a node

This is the spine of the entire book. Part II walks the code in roughly the order events happen here, so internalize this narrative now. We tell it in two acts: **startup** (the node assembling itself and getting ready) and **steady state** (the node running and doing its job). There is no code in this section — just the story.

Act I — Startup

Picture an operator typing this into a terminal:

```
hathor-cli run_node --testnet --data ./data
```

1. The command is dispatched. The `hathor-cli` program looks at the first word after it — `run_node` — and routes to the code responsible for that subcommand. (Internally this is a dictionary lookup from command name to a Python module; we cover it in Chapter 21.) There are roughly forty subcommands; `run_node` is the one that boots an actual node.

2. Arguments and logging are set up. The flags (`--testnet`, `--data ./data`, and dozens of optional ones) are parsed into a structured configuration object, and structured logging⁸ is switched on so that everything the node does from here is recorded.

3. Settings are loaded. The node must know which network it is joining — `mainnet` (real money), `testnet` (a practice network), or a private network. This choice selects a **settings profile**: a file of constants (the genesis data, timing parameters, feature schedules) that every later step reads. Get this wrong and the node would be speaking a different dialect than its peers. (Chapter 22.)

4. The reactor is initialized. Hathor is built on a framework called **Twisted**, whose centerpiece is the **reactor**⁹ — the single event loop¹⁰ that will drive the whole program once it is running. At startup the reactor is created but not yet running; think of it as starting the car's electrical system before putting it in gear. Twisted and the reactor are so central that they get a full chapter early (Chapter 16) and an abstraction chapter of their own (Chapter 23).

5. The node is assembled — the "builder" phase. This is the most intricate part of startup. A builder¹¹ constructs, in the right order, every component the node needs and wires them together:

- the **storage** layer opens the on-disk database (RocksDB);
- the **indexes** are prepared (fast lookup tables for balances, addresses, heights, and more);
- the **wallet** is loaded, if one is configured;
- the **pub-sub**¹² system and **event manager** are created (the node's internal announcement system);
- the **verification** and **consensus** services are built;
- the **nano-contract** runtime and the **feature-activation** service are set up.

The result is a single object, the `HathorManager`, holding references to all of these. The manager is the node's central coordinator — the closest thing to a "main object." It is important enough to get its own chapter (Chapter 29), and assembling it is important enough to get another (Chapter 24).

6. The manager starts. Now the assembled node is told to start. Several things happen in a deliberate order:

- **Crash check.** The node inspects a flag in storage. If the last shutdown was not clean, the on-disk bookkeeping may be untrustworthy, and the node refuses to start rather than risk acting on corrupt data. (This is a real safety mechanism, not a formality.)
- **Initialization.** The node loads the **genesis**¹³ — the hard-coded first block and transactions that every node agrees on as the starting point of history — and rebuilds its in-memory view of the ledger from what is already on disk. During this phase the storage is told to allow access to data in any validation state; once initialization finishes, access is narrowed to only fully-valid data. (Why the node temporarily relaxes and then tightens this "scope" is a subtle point we return to in Chapter 25.)
- **Subsystems come online**, again in order: the WebSocket server (if enabled), metrics collection, then the **peer-to-peer connections manager**, then the wallet, then the mining server (if mining is enabled), then the proof-of-authority block producer (only on certain private networks).
- Finally the manager marks itself **READY**.

7. The reactor is put in gear. The reactor's event loop is told to run. Control passes to Twisted, and the startup story ends. From here the node is reactive: it sleeps until something happens — a peer connects, a transaction arrives, a timer fires — and responds.

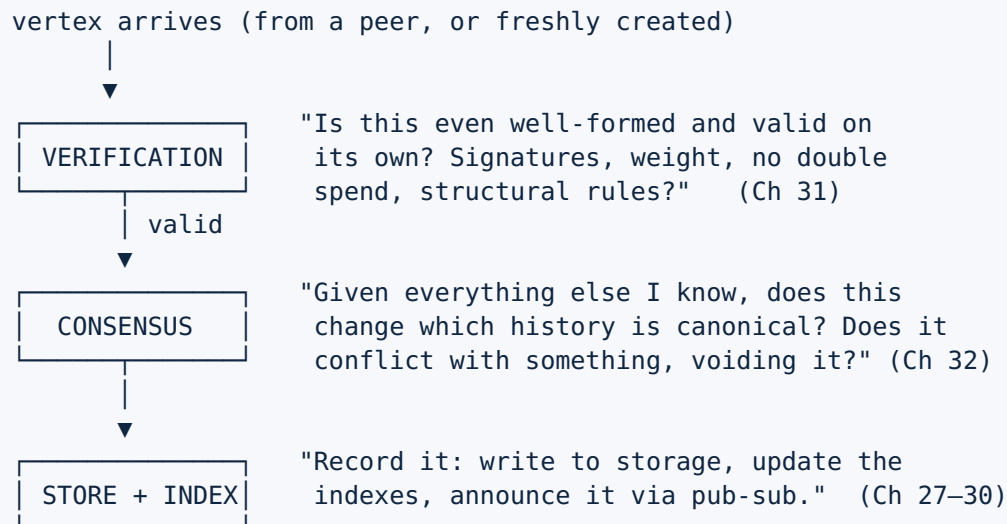
Act II — Steady state

Now the node is alive. Four kinds of activity run concurrently, all driven by the one reactor:

Finding and connecting to peers. The connections manager discovers other nodes (from a bootstrap list and from peers themselves) and opens connections. Each connection performs a handshake in which the two nodes agree on a protocol version and exchange who they are. (Chapter 34.)

Syncing. A freshly started node is usually far behind — it may have a near-empty database while the network has years of history. Over each peer connection, a sync agent negotiates and downloads the blocks and transactions the node is missing, in dependency order, until it has caught up. Hathor's current protocol for this is **sync-v2**¹⁴; an older sync-v1 has been removed from the codebase entirely, so any old comment mentioning it is stale. (Chapter 35.)

Ingesting new vertices. Whether a vertex arrives from syncing or as a brand-new transaction broadcast across the network, it flows through the same pipeline:



The component that runs this pipeline end-to-end is the **vertex handler** (Chapter 33). It is a small but pivotal piece: it is where "data arrived" becomes "ledger changed."

Serving clients. Concurrently, the node answers wallets over HTTP, streams events over WebSockets, and — if mining is on — hands out work to miners and accepts the blocks they find. A miner repeatedly tries to solve a proof-of-work¹⁵ puzzle; when it succeeds, the resulting block re-enters the ingestion pipeline above just like any other vertex. (Chapter 37.)

That is the whole life of a node, start to finish. Every chapter in Part II is a zoom-in on one box in this story.

0.4 The module map

`hathor-core` lives in two top-level places: the package `hathor/` (the node itself) and the package `hathor_cli/` (the command-line surface that launches and operates it). Inside `hathor/` are roughly two dozen sub-packages. Rather than list them alphabetically, here they are grouped by the five responsibilities from §0.2 plus the infrastructure that supports them.

hathor-core/	
├─ hathor_cli/	← command-line surface (run_node + ~40 tools)
├─ hathor/	
│ manager.py	← HathorManager: central coordinator ◀ YOU ARE HERE
│ vertex_handler/	← ingestion pipeline (verify→consensus→store)
├─ — DOMAIN MODEL —	
│ transaction/	← vertex model: Block, Transaction, in/outputs, metadata
│ serialization/	← the bespoke binary wire format
│ verification/	← rule-checking for vertices
│ consensus/	← which history is canonical; weights, voiding, PoA
├─ — STORAGE & LOOKUP —	
│ storage/	← low-level RocksDB key-value wrapper
│ transaction/storage/	← vertex-aware storage on top of RocksDB
│ indexes/	← fast lookups: UTX0, address, tokens, height, mempool
├─ — NETWORKING & CLIENTS —	
│ p2p/	← peer-to-peer protocol, connections, sync-v2
│ websocket/	← admin / streaming WebSocket surface
│ event/ pubsub.py	← internal + external event system
├─ — MINING & MONETARY —	
│ mining/	← block templates, CPU miner service
│ stratum/	← the Stratum mining protocol
│ merged_mining/	← mine Hathor as auxiliary work to Bitcoin
│ daa.py difficulty.py	← difficulty adjustment & weight math
│ reward_lock.py	← rules locking freshly-mined rewards
├─ — HIGHER SERVICES —	
│ nanocontracts/	← smart-contract runtime (largest subsystem)
│ feature_activation/	← miner-voted protocol upgrades
│ wallet/ crypto/ pycoin/	← key management & signing
├─ — INFRASTRUCTURE —	
│ builder/	← assembles & wires the node
│ reactor/	← Twisted reactor abstraction
│ conf/	← settings & network profiles
│ sysctl/	← runtime control socket
│ metrics.py profiler/ healthcheck/	← observability
│ simulator/ dag_builder/	← deterministic testing infrastructure
│ execution_manager.py	← task coordination

What follows is one paragraph per major area — enough to know what each is for. The depth comes later.

Domain model — the data and its rules

transaction/ is the heart of the data model. Despite the name, it defines all vertex types: the base class `BaseTransaction`, and its subclasses `Block`, `Transaction`, and `MergeMinedBlock`, along with their inputs, outputs, and the per-vertex **metadata**¹⁶ the

node maintains (height, accumulated weight, whether the vertex has been voided, and more). It is one of the largest and most foundational packages; the data structures it defines are referenced everywhere. (Chapter 25.)

serialization/ turns those vertices into bytes and back. Hathor does **not** use JSON or Protobuf for its wire format; it has a custom, compact binary encoding. Whenever a vertex is sent to a peer or written to disk, this package does the translation. (Chapter 26.)

verification/ holds the rules. For each vertex type there is a verifier that checks the vertex obeys the protocol — structurally and cryptographically — in isolation. (Chapter 31.)

consensus/ decides canonical history. When vertices conflict, or when there are competing chains of blocks, the consensus algorithm uses accumulated **weight**¹⁷ to decide which version wins and marks the losers as **voided**. It also contains the proof-of-authority variant used by certain private networks. (Chapter 32.)

Storage and lookup — where the ledger lives

storage/ is a thin wrapper around **RocksDB**¹⁸, an embedded key-value database. It speaks in raw bytes: put this key, get that key. **transaction/storage/** sits on top and speaks in vertices: store this block, fetch the transaction with this hash, walk the graph from here. **indexes/** maintains the derived lookup tables that make common questions fast — which outputs are unspent (UTXO), what is this address's history, what is the current height, what is waiting in the mempool¹⁹. Indexes can be rebuilt from the stored vertices, which is part of what the initialization phase in §0.3 does. (Chapters 27–28.)

Networking and clients — talking to the world

p2p/ is among the largest packages. It implements the peer-to-peer protocol: discovering peers, establishing and managing connections (each connection is driven by a Twisted protocol object and progresses through a state machine), and running sync-v2 to exchange ledger data. **websocket/** exposes a streaming interface for dashboards and integrations. **event/** and **pubsub.py** form the announcement system: internally, components subscribe to events like "a new block was accepted"; externally, an optional persistent event queue lets downstream systems replay everything the node has seen. (Chapters 30, 34–36.)

Mining and monetary — minting and securing coins

mining/ produces block templates (the puzzle a miner must solve) and includes a simple CPU miner. **stratum/** implements the **Stratum** protocol that real mining software uses to talk to the node. **merged_mining/** lets miners mine Hathor blocks as a

side-effect of mining Bitcoin. `daa.py` and `difficulty.py` hold the difficulty-adjustment math that keeps blocks arriving at a steady rate. `reward_lock.py` enforces that freshly-mined coins can't be spent for a while. (Chapter 37.)

Higher services — built on the base ledger

`nanocontracts/` is the smart-contract engine and the single biggest subsystem (over fifteen thousand lines). It defines blueprints (contract templates), a runner that executes contract code with metered resource limits²⁰, and its own state storage. `feature_activation/` governs protocol upgrades: new features are switched on by miner signalling²¹ over a schedule, so the network can change rules without a flag day. `wallet/`, `crypto/`, and `pycoin/` handle keys, addresses, and digital signatures. (Chapters 38–40.)

Infrastructure — making it all run

`builder/` assembles the node (§0.3, step 5). `reactor/` wraps Twisted's reactor so the node can optionally run on an asyncio backend instead. `conf/` loads the settings profiles. `sysctl/` is a control socket for poking a running node (query state, tweak parameters). `metrics.py`, `profiler/`, and `healthcheck/` are observability. `simulator/` and `dag_builder/` let tests spin up a deterministic, in-memory network — invaluable for reproducing bugs. (Chapters 23–24, 41–43.)

The command-line surface

`hathor_cli/` is everything you invoke from a terminal: the dispatcher that routes subcommands, `run_node` itself, and a toolbox of operator and developer utilities (generate a wallet, export the database, run a miner, dump nano-contract state, and so on). It is the outermost layer — the front door — and so it is the first package we open in Part II. (Chapter 21.)

0.5 How to read this book

This book is written for a developer who is comfortable writing Python but has not yet met most of the senior-level concepts, surrounding tooling, or blockchain theory that `hathor-core` assumes. So it teaches three kinds of thing, woven together:

- **Programming concepts** — the computer-science scaffolding: object orientation, callbacks, asynchronous programming, design patterns, decorators, and the like. (Part I, Chapters 1–5.)
- **Domain concepts** — blockchain and Hathor theory: the DAG, the UTXO model, weight and consensus, checkpoints. (Part I, Chapters 6–10.)

- **The stack** — the third-party technologies the node is built from: Poetry, Docker, Twisted, RocksDB, and more. (Part I, Chapters 11–20, and at the modules that use them.)

Three reading aids

To keep the main text readable while still defining everything, the book uses three devices consistently. Recognize them and you will know how much detail to expect:

1. **Footnotes** define a single word or piece of jargon on the spot, in a sentence or two. Every footnoted term also appears in the glossary at the back. This page is full of them — scroll to the bottom to see the style.
2. **Recap boxes** look like this:

Recap — vertex (full treatment in Ch. 8 & 25). A vertex is any node in Hathor's ledger graph — either a [Block](#) or a [Transaction](#). The codebase uses "vertex" as the umbrella term when it does not care which kind it is holding. We will define it properly, with the graph theory behind it, in Chapter 8, and meet it in code in Chapter 25.

They re-introduce a concept in the context of the chapter you are reading, just enough to follow along, and point you to the full treatment elsewhere. They exist because meeting an idea again in a new setting is how it sticks — so do not skip them as repetition.

1. **Deep-dives** are the full, once-per-concept treatment — anywhere from two to twenty pages — placed where the concept first truly matters. Each technology and each concept is explained in depth exactly once; everywhere else you get a footnote or a recap box pointing back to it.

The order, and how to navigate

Part I builds your vocabulary before you touch the codebase, concepts first, then the tools. Part II then walks the node in the order of the life-of-a-node story from §0.3, beginning at the command line and ending with the testing infrastructure. You can read straight through, and that is the intended path. But you can also use §0.3 and §0.4 as an index: find the box or package you care about, follow its chapter pointer, and read the recap boxes there to pull in whatever background you are missing.

One convention worth stating plainly: when this book cites code, it gives the file path always, and a line number when it names a specific identifier — like `manager.py:276`. Those are clickable in the source markdown and precise enough to grep. The line numbers are accurate as of the branch named on the cover; if the code has moved since, treat them as close-but-verify.

Recap

Responsibility (§0.2)	Where it lives (§0.4)	Covered in
Store the ledger	<code>storage/</code> , <code>transaction/storage/</code> , <code>indexes/</code>	Ch 27–28
Validate everything	<code>verification/</code>	Ch 31
Reach consensus	<code>consensus/</code> , <code>daa.py</code> , <code>difficulty.py</code>	Ch 32
Talk to peers (sync)	<code>p2p/</code>	Ch 34–35
Serve clients	<code>websocket/</code> , <code>event/</code> , <code>mining/</code> , <code>stratum/</code>	Ch 30, 36–37
Coordinate it all	<code>manager.py</code> , <code>vertex_handler/</code> , <code>builder/</code>	Ch 24, 29, 33
The data itself	<code>transaction/</code> , <code>serialization/</code>	Ch 25–26
Higher services	<code>nanocontracts/</code> , <code>feature_activation/</code> , <code>wallet/</code>	Ch 38–40
Launch & operate	<code>hathor_cli/</code>	Ch 21

You now have the two things this chapter exists to give you: the **life of a node** (§0.3) — the order in which everything happens — and the **module map** (§0.4) — where each piece of that story lives in the codebase. Every remaining chapter is a zoom-in on one box in that story or one tool it depends on. The next part of the book steps back from Hathor entirely to build the programming and blockchain vocabulary the code assumes; if you are impatient to see code, you can jump ahead to Chapter 21 and lean on the recap boxes — but the foundations are there because the code will make far more sense with them in place.

-
1. A ledger is just a record of transactions and balances — historically an accountant's book. A distributed ledger is one copy of which is held and maintained by many computers at once, with no single authoritative master copy. ↩
 2. **DAG** = Directed Acyclic Graph. A graph is a set of items ("nodes") joined by links ("edges"). Directed means each edge points one way. Acyclic means you can never follow the edges in a loop back to where you started. Full treatment in Chapter 8. ↩
 3. Vertex is the standard graph-theory word for a node of a graph. Hathor's code uses it as the common term for "a block or a transaction," since both are nodes of the ledger DAG. ↩
 4. A smart contract is a small program stored on a ledger that automatically moves tokens according to its own rules when called — e.g. "release these funds only if two of three people approve." Hathor's version is called a nano-contract. ↩

5. A daemon is a program that runs continuously in the background rather than finishing and exiting. Web servers and databases are daemons. The word is pronounced "demon" and comes from an old Unix tradition. ↵
6. Peer-to-peer (P2P) means every participant is an equal "peer" that connects directly to other peers, with no central server in the middle. Contrast a client-server model, where everyone talks through one company's servers. ↵
7. Syncing (synchronizing) is the process by which a node that is behind downloads the blocks and transactions it is missing from its peers until its copy of the ledger matches the network's. ↵
8. Structured logging records log entries as machine-readable key-value data (e.g. `event="block accepted" height=42`) rather than as freeform sentences, so the logs can be searched and analyzed programmatically. Hathor uses a library called `structlog`; full treatment in Chapter 17. ↵
9. The reactor is the central object of the Twisted framework: a single loop that waits for events (network data, timers, etc.) and calls the right piece of your code in response. Full treatment in Chapter 16. ↵
10. An event loop is a program structure that repeatedly waits for "something to happen" and then dispatches a handler for it, instead of running top-to-bottom and blocking. It is the core idea behind asynchronous programming — Chapter 2. ↵
11. A builder is a design pattern: an object whose job is to construct another, complicated object step by step. Hathor's builder constructs the fully-wired node. The pattern is explained in Chapter 3; Hathor's builders in Chapter 24. ↵
12. Pub-sub (publish-subscribe) is a messaging pattern where publishers announce events without knowing who is listening, and subscribers register interest in event types. It decouples the announcer from the reactors. Chapter 30. ↵
13. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions — that every node agrees on by definition. Without a shared genesis, two nodes would have no common root of history. ↵
14. Sync-v2 is the current peer-to-peer synchronization protocol. An earlier sync-v1 existed but has been removed; only sync-v2 remains. Chapter 35. ↵
15. Proof-of-work (PoW) is a scheme where producing a valid block requires finding a number that makes the block's hash fall below a target — work that is hard to do but trivial to check. It is what makes rewriting history expensive. Chapter 9. ↵
16. Metadata here means data the node computes and stores about a vertex (its height, accumulated weight, voided status, ...) as opposed to the vertex's own fixed contents. It is recomputed and updated as the ledger evolves. Chapter 25. ↵
17. Weight is a numeric measure of how much proof-of-work a vertex represents; accumulated weight sums the weight reachable through the graph. Consensus prefers the history with the most accumulated weight. Chapter 9 & 32. ↵
18. **RocksDB** is an embedded key-value database: a fast on-disk store of `key → value` byte pairs that runs inside the node's own process rather than as a separate server. Full treatment, and the comparison against alternatives like MongoDB, in Chapter 27. ↵
19. The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed/ordered by a block. Chapter 28. ↵
20. Metered execution means the contract runner counts the resources a contract uses (a kind of "gas") and stops it if it exceeds a limit, so a buggy or malicious contract cannot run forever. Chapter 39. ↵
21. Miner signalling is a voting mechanism: miners set specific bits in the blocks they produce to indicate support for a proposed upgrade, and the rule change activates once enough support accumulates over a defined window. Chapter 38. ↵